

Zurn Elkay (zurnelkay-eds-ue) - Developer Guide

Date: September 23, 2026



Table of Contents

Zurn Elkay (zurnelkay-eds-ue) - Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development Setup	6
Design System	7
Blocks Reference	8
Shared Utilities (scripts/utls.js)	10
Integrations (delayed.js + head.html)	10
Common Development Tasks	11
Environments	11
Git Workflow	12
Troubleshooting	12
Resources	13
Support Contacts	13

Zurn Elkay (zurnelkay-eds-ue) - Developer Guide

This guide is the technical handover for the Zurn Elkay AEM Edge Delivery Services project. It documents the architecture, custom code, design system, and the migration-specific blocks that were built for this site. It is written for developers who will maintain and extend the project.

Quick Reference

Resource	URL
Code Repository	oharris69/zurnelkay-eds-ue (GitHub, from Config Service)
Code source URL	https://github.com/oharris69/zurnelkay-eds-ue
Preview	https://main-zurnelkay-eds-ue-oharris69.aem.page/
Live	https://main-zurnelkay-eds-ue-oharris69.aem.live/
Local Dev	http://localhost:3000
Config Service org	oharris69

Project type: AEM Universal Editor / crosswalk (xwalk) project. Content is authored in AEM (Universal Editor) and delivered as markup, not from a document source. This is confirmed by the Config Service content source and the presence of `component-definition.json`, `component-models.json`, `component-filters.json`, and the `models/` folder.

Content source (from Config Service): `https://author-p7954-e2285674.adobe.aemcloud.com/bin/franklin.delivery/oharris69/zurnelkay-eds-ue/main` (type markup, suffix `.html`).

This is a **standard single-site setup** — the Config Service returns a single site (`zurnelkay-eds-ue`) whose code lives in the GitHub repo of the same name. There is no repoless/multi-site configuration for this project.

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+), no framework
- CSS3 with Custom Properties (design tokens)
- No build step for blocks/scripts — files are served directly
- A build step exists only for the Universal Editor model JSON (see below)
- Content authored in AEM via the Universal Editor, delivered as markup

Universal Editor Model Build

Unlike document-based projects, this crosswalk project generates its component JSON from source parts in `models/`. The `package.json` scripts merge those parts into the root JSON files that AEM consumes:

```
npm run build:json          # runs all three below in parallel
npm run build:json:models    # models/_component-models.json    -> component-
```

```
models.json
npm run build:json:definitions # models/_component-definition.json -> component-
definition.json
npm run build:json:filters      # models/_component-filters.json      -> component-
filters.json
```

Important: After changing a block's authoring model, edit the source under `models/` and re-run `npm run build:json`. Do not hand-edit the root `component-*.json` files — they are generated artifacts.

Project Structure

```
├── blocks/           # 43 block folders (mix of boilerplate + custom migration blocks)
├── models/           # Universal Editor model source parts (built into component-*.json)
├── scripts/
│   ├── aem.js        # Core library (DO NOT MODIFY)
│   ├── scripts.js    # Custom page decoration, E-L-D orchestration, DM/Scene7 auto-block
│   ├── delayed.js    # Adobe Target, custom libraries, tweetable links
│   ├── utils.js      # Shared multi-language + Dynamic Media helpers
│   ├── dom-helpers.js # Hyperscript-style DOM builders (div, a, img, span, i, ...)
│   ├── experiment-load.js # Experimentation plugin loader
│   └── ...           # slider.js, ffetch.js, placeholders.js, editor-support*.js, etc.
├── styles/
│   ├── styles.css    # Critical styles + design tokens
│   ├── brand.css     # Zurn brand token overrides
│   ├── fonts.css     # Font-face declarations
│   ├── lazy-styles.css # Non-critical styles
│   └── slider.css, card-image-zoom.css
├── fonts/           # Adobe Clean, Roboto, Font Awesome font files
├── icons/           # SVG icons
├── head.html        # <head> injections (favicons, Scene7, Adobe Launch, DM video)
├── component-*.json # Generated Universal Editor model artifacts
└── models/          # Source for the above
```

Three-Phase Loading (E-L-D)

AEM Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. Every file and script must be placed in the correct phase.

Phase	Purpose	What Loads Here	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	styles/styles.css, first section's blocks, scripts/scripts.js, language + theme decoration, experimentation	Blocks LCP — keep minimal
Lazy	Load remaining page content after first paint	Remaining sections/blocks, header, footer, fonts.css, lazy-styles.css	Runs after Eager — safe for non-critical UI
Delayed	Load non-essential third-party scripts	scripts/delayed.js (Adobe Target, custom libraries) — imported ~3s after load	Never blocks rendering

Rules for developers: - Never add third-party/marketing scripts to `scripts.js` — they block LCP. Use `delayed.js`.
 - Fonts load lazily (and conditionally in eager only on desktop / when already cached) — do not force eager font loading.
 - Blocks in the first section load eagerly; all others load lazily, automatically based on DOM position.
 - Header and footer are loaded in the lazy phase via `loadHeader()` / `loadFooter()`.

Project-Specific Loading Details (scripts.js)

scripts/scripts.js is heavily customized for this project. Key additions beyond the boilerplate:

- **loadPage() flow:** loadEager(document) → loadLazy(document) → loadDelayed(). loadDelayed() imports delayed.js after a 3s setTimeout.
- **loadEager** calls setPageLanguage() (multi-language), decorateTemplateAndTheme(), runExperimentation(), and renderWBDataLayer() (a window.wbgData data layer) before decorating the first section.
- **loadLazy** loads sections, header, footer, lazy-styles.css, fonts, and shows the experimentation rail.
- **Dynamic Media / Scene7 auto-block** (marked `--- BEGIN DM/Scene7 auto-block (excavated) ---`): a large body of code (detectDynamicMediaUrl, buildScene7Rendition, buildDmOpenApiRendition, renderScene7Picture, renderDmOpenApiPicture, buildDynamicMediaImages, decorateDMImages, isDMOpenAPIUrl) that detects Adobe Dynamic Media / Scene7 image links in authored content and rewrites them into responsive <picture> elements with breakpoint-specific renditions (desktop min-width: 600px @ 2000px, mobile @ 750px).
- **decorateButtons** is customized (runs before buildAutoBlocks in decorateMain) and handles unwrapping markdown-style link paragraphs.
- **isAuthorEnvironment()** is exported and consumed by header.js and footer.js to branch behavior inside the Universal Editor.
- **fetchLanguagePlaceholders()** loads placeholders scoped to the current language via getLanguage().

External scripts in **head.html** (loaded outside the E-L-D JS phases): Scene7 responsive image viewer, Adobe Launch (DTM) tag manager (async), and the Adobe DM video viewer. Favicons point at www.zurn.com.

Key Files & How They Connect

File	Role	Connects To
scripts/aem.js	Core library — loadBlock, loadCSS, decorateBlock, decorateSections, sampleRUM, loadHeader/loadFooter, createOptimizedPicture, etc. DO NOT MODIFY.	Imported by scripts.js, utils.js, and blocks
scripts/scripts.js	Entry point — E-L-D orchestration, decorateMain, buildAutoBlocks, DM/Scene7 auto-block, data layer.	Imports from aem.js, utils.js, experiment-load.js
scripts/utils.js	Multi-language + Dynamic Media helpers (see below).	Imported by scripts.js, header.js, footer.js, and other blocks
scripts/delayed.js	Adobe Target loader, external custom-library embedding, tweetable-link autoblock.	Imported by scripts.js after 3s
scripts/dom-helpers.js	Hyperscript-style element builders.	Imported by delayed.js, header.js, footer.js, blocks
styles/styles.css	Critical CSS + design tokens. Loaded eagerly via head.html.	Consumes tokens from brand.css
styles/brand.css	Zurn brand token overrides (colors, fonts, sizes).	Feeds --brand-* tokens into styles.css

File	Role	Connects To
styles/fonts.css	@font-face declarations (Adobe Clean, Roboto, Font Awesome). Loaded lazily.	Font families referenced in styles.css/brand.css
blocks/{name}/{name}.js	Block logic — exports default function decorate(block).	May import utils.js, aem.js, dom-helpers.js
models/_component-*.json	Universal Editor authoring model source.	Built into root component-*.json via npm run build:json

Local Development Setup

Prerequisites

- Node.js (no .nvmrc or engines pin in this repo — use the current AEM-supported LTS)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
# Clone repository
git clone https://github.com/oharris69/zurnelkay-eds-ue
cd zurnelkay-eds-ue

# Install dependencies
npm install

# Start local server
aem up
```

Local Server

- URL: `http://localhost:3000`
- Auto-reload on file changes
- Uses preview content by default

This is a standard single-site project, so `aem up` does not need a `--pages-url` flag.

Editing Authoring Models

```
# After changing anything under models/, regenerate the root component JSON:
npm run build:json
```

Linting

```
npm run lint      # runs both of the below
npm run lint:js   # eslint . --ext .json,.js,.mjs (airbnb-base + xwalk plugin)
npm run lint:css  # stylelint on blocks/**/*.css and styles/*.css
```

A Husky pre-commit hook is configured (prepare: husky).

Design System

The design system is split into two files: **brand.css** holds Zurn-specific brand overrides (`--brand-*` tokens) extracted from zurn.com, and **styles.css** consumes those tokens with fallbacks. To re-theme, prefer editing **brand.css**.

Brand Tokens (brand.css)

```
--brand-background-color: #fff;
--brand-text-color: #1f262e;      /* Zurn soft near-black body text */
--brand-link-color: #005995;      /* Zurn blue */
--brand-link-hover-color: #0a58ca;
--brand-theme-color: #005995;
--brand-dark-color: #1e2f3d;      /* dark resources band */
--brand-light-color: #f2f2f2;
--heading-primary-color: #1b4462; /* Zurn deep navy for headings */
--brand-blue: #005995;
--brand-blue-bright: #0075c9;      /* Zurn wordmark blue */
--brand-nav-slate: #1b2a34;
--brand-nav-accent-start: #00b497; /* teal accent */
--brand-nav-accent-end: #0368ad;   /* blue accent */
--brand-brandbar-color: #3f3f3f;   /* charcoal top brand bar */
--brand-nav-background-color: #f6f6f6; /* light-grey logo/search row */
--brand-nav-text-color: #1b2a34;
```

Core Tokens (styles.css)

Typography

```
--body-font-family: "Gotham Zurn", Arial, Helvetica, sans-serif;
--heading-font-family: "Gotham Zurn", Arial, Helvetica, sans-serif;
--body-font-size-m: 22px; /* s: 19px, xs: 17px (desktop) */
--heading-font-size-xxl: 55px; /* down to xs: 22px (desktop) */
```

Note: "Gotham Zurn" is a proprietary font with no public web source, so the stack falls back to Arial/Helvetica — matching zurn.com's own fallback.

Colors (resolved from brand tokens)

```
--background-color: var(--brand-background-color, #fff);
--text-color: var(--brand-text-color, #131313);
--link-color: var(--brand-link-color, #131313);
--link-hover-color: var(--brand-link-hover-color, #F97316);
--main-accent-color: var(--brand-theme-color, #FB923C);
--nav-background-color, --nav-text-color, --footer-background-color, --footer-text-color
```

Spacing & Layout

```
--global-gutter: 1.5rem;
--global-section-padding: 5rem;
--grid-margin: 4.5rem;
--spacing-none ... --spacing-xhuge: 0 ... 6rem; /* full spacing scale */
--border-radius-none ... --border-radius-x-large: 0 ... 2rem;
```

```
--nav-height: 64px; /* overridden to 34px in brand.css */
--section-gutter-space: 28px;
```

Fonts

Family	Weights	Source Files	Usage
Adobe Clean (+ Light/ Bold/Extra Bold/Black)	300–900	AdobeClean*.woff	Adobe-brand text
Roboto	400 / 500 / 700	roboto-*.woff2	Secondary text
Roboto Condensed	700	roboto-condensed-bold.woff2	Condensed headings
Font Awesome 5 (Brands/Free/Solid)	400 / 900	fonts/fa/*	Icons

Note: the design falls back to “Gotham Zurn”/Arial for the brand type; the bundled Adobe Clean/Roboto faces are declared in `fonts.css`.

Breakpoints

Standard AEM Edge Delivery Services breakpoints are used (no unusual deviations found in `styles.css`):

Name	Min-Width	Usage
Mobile	0	Default styles
Tablet	600px	@media (min-width: 600px)
Desktop	900px	@media (min-width: 900px) — also the desktop nav / eager-font threshold
Large	1200px	@media (min-width: 1200px)

Section Styles

Available section modifiers applied via section metadata:

- `.section.light`, `.section.highlight`, `.section.dark` — background/theme variants (dark inverts heading and text colors).
- Data-attribute spacing scale: `data-sec-spacing="section-{none|xtiny|tiny|xxsmall|xsmall|small|regular|medium|large|xlarge|xxlarge|huge|xhuge}"` and matching `data-sec-spacing-bottom="..."` for per-side control.
- `data-sec-full-width="true"` — full-bleed section.

Blocks Reference

This project mixes the AEM boilerplate block set with a set of **custom migration blocks** built to reproduce Zurn’s site. The custom blocks below were authored during the migration (each carries a descriptive migration commit and a documented decorator contract); they are the ones worth understanding for maintenance. Standard boilerplate blocks (accordion, cards, carousel, columns, hero, tabs, video, quote, embed, fragment, search, etc.) follow their normal AEM behavior and are not re-documented here.

Design convention: the custom block JS files are **structural decorators only** — they build the DOM/ARIA structure and add class hooks. Brand colors and visual styling live in the block CSS and the design tokens, not in the JS.

Block	Purpose	Notes / Variants
header	Site header/nav. Loads a nav fragment, wires Zurn logos, multi-language switch, brand bar, and search box. Most-iterated block (9 commits).	Uses <code>getSiteName</code> , <code>computeLocalizedUrl</code> , <code>discoverLanguagesFromPlaceholders</code> ; <code>isDesktop</code> matches min-width: 900px; branches on <code>isAuthorEnvironment()</code> .
footer	Site footer. Loads a footer fragment and swaps the text brand line for the real Zurn footer logo from the DAM.	<code>FOOTER_LOGO = /content/dam/zurn/en/brand/zurn-footer-logo-bottom.svg</code> ; language-aware.
carousel-hero	Full-bleed rotating hero banner; each row = one slide (bg image + heading/paragraph/CTA). Prev/next arrows, dots, optional autoplay.	First config row can set <code>data-autoplay</code> . Tracks instance count for unique IDs.
carousel-markets	Rotating carousel of market cards (image + name + description + link); multi-card view on desktop.	Renders <code>/</code> track with paging arrows.
carousel-media	Innovation media-tile carousel (image + linked label per tile), horizontally paged.	Intro heading/CTA sits as section default content, not inside the block.
cards-feature	Three-up feature card grid (icon + title + description + photo).	Detects photo-only cells and tags them <code>cards-feature-photo</code> .
cards-icon	Four-up centered icon feature row (icon + title + short description).	Detects icon-only cells and tags them <code>cards-icon-icon</code> .
tabs-audience	Centered text tabs with image+heading+text panels; row 1 cell = label, rest = panel.	ARIA <code>tablist</code> ; instance-counted IDs.
tabs-resource	Tabbed resource switcher; each tab holds a grid of resource-link cards.	ARIA <code>tablist</code> ; instance-counted IDs.
hero-market	Full-bleed vertical-market hero: bg image + bottom-left overlay heading/subheading.	Single image cell + text cell.
columns-band	Full-bleed colored media/text band (image + heading/copy/CTA).	Variants via <code>classList</code> : <code>image-right</code> (default <code>image-left</code>), <code>dark</code> , <code>teal</code> .
columns-stat	Two-column stat callout (large figure+label / supporting paragraph).	Tags cells <code>columns-stat-figure</code> / <code>columns-stat-body</code> .
columns-product	Product-detail two-column layout (product image / SKU eyebrow, title, description, "Where to Buy" CTA, feature list). Optional badge row.	Adds <code>columns-product-{n}-cols</code> class based on column count.
breadcrumb-product	Horizontal product-hierarchy breadcrumb trail; linked crumbs = ancestors, plain text = current page.	Renders semantic <code><nav></code> with / delimiters.
download-resources	Dark "Resources" band listing downloadable document links with a download icon.	Rows: <code>[link]</code> or <code>[group label link]</code> ; leading text-only row = subgroup heading. Inline SVG icon.

Block	Purpose	Notes / Variants
download-list	Boilerplate-derived downloadable file list.	Standard list rendering.

Block Dependencies

- `header.js` / `footer.js` import from `aem.js`, `utils.js`, `dom-helpers.js`, `../fragment/fragment.js`, and `scripts.js` (`isAuthorEnvironment`).
- Most custom content blocks (carousels, cards, columns, tabs) are self-contained structural decorators with no external imports.
- Dynamic Media rendering for authored images is handled centrally in `scripts.js` (not per-block).

Shared Utilities (`scripts/utils.js`)

`utils.js` is a substantial custom module centered on **multi-language** support and **Dynamic Media**. Key exports:

Export	Purpose
<code>PATH_PREFIX (/language-masters), SUPPORTED_LANGUAGES, TAG_ROOT, INTERNAL_PAGES</code>	Multi-language + internal-page constants
<code>getLanguage(), setPageLanguage(), getPathDetails()</code>	Detect/set the current page language
<code>computeLocalizedUrl(targetLang), discoverLanguagesFromPlaceholders()</code>	Language switching for the header
<code>fetchLanguageNavigation(langCode), getPathMappings(), mapAemPathToSitePath(aemPath)</code>	Resolve AEM authoring paths to site paths per language
<code>getHostname(), getHref(), isInternalPage()</code>	Environment/URL helpers
<code>getDynamicMediaServerURL(), dynamicMediaAssetProcess(pictureElement, qParam), createSource(src, width, mediaQuery)</code>	Dynamic Media rendering support
<code>getInheritedPageProperties(), processTags(tag, prefix), formatDate(dObjStr), fetchData(...)</code>	Page metadata, tags, formatting, fetch

The English/French bilingual structure (`/fr/...` pages, language-master paths) is a defining characteristic of this project — several migration blocks and the header/footer were built to be language-aware.

Integrations (`delayed.js` + `head.html`)

Loaded in the **delayed** phase (`delayed.js`, ~3s after load): - **Adobe Target** — `loadAT()` loads `/scripts/at-lsig.js`; `at_property = 549d426b-0bcc-be60-ce27-b9923bfcad4f`. `window.targetGlobalSettings.bodyHidingEnabled = false`. Skipped inside `/canvas/` (author) contexts and on localhost. - **Custom external libraries** — `embedCustomLibraries()` reads a comma-separated `js-files` page metadata value and `loadScript()`s each URL. - **Tweetable links** — `buildTwitterLinks()` finds `<tweetable>` markup in paragraphs and turns it into Twitter share popups.

Loaded via **head.html** (outside the JS phases): - Scene7 responsive image viewer (`s7viewers/libs/responsive_image.js`, `defer`) - Adobe Launch / DTM tag manager

(assets.adobedtm.com/...launch-...development.min.js, async) — note this is the **development** Launch library; verify the correct environment library before go-live. - Adobe DM video viewer (DMVideoViewer.js) - Zurn favicons served from www.zurn.com

There is no Google Analytics/gtag or OneTrust integration in this project; analytics/tag management flows through Adobe Launch + Adobe Target.

Common Development Tasks

Add a New Block

1. Create blocks/{name}/{name}.js and blocks/{name}/{name}.css.
2. Implement export default function decorate(block) (structural DOM only; keep brand styling in CSS).
3. If it is author-editable, add the model to models/_component-*.json and run npm run build:json.
4. Test locally at http://localhost:3000; run npm run lint.
5. Push to a feature branch.

Modify Global Styles / Re-theme

1. For brand colors/fonts, edit styles/brand.css (--brand-* tokens).
2. For structural styles, edit styles/styles.css.
3. Test across multiple pages and both / and /fr/ locales; watch for CLS.

Add Analytics/Marketing Tool

1. Add it to scripts/delayed.js (or as a Launch rule). Never add to scripts.js.
2. Verify in the Network tab that it loads ~3s after page load.

Change an Authoring Model

1. Edit the source under models/ (not the root component-*.json).
2. Run npm run build:json.
3. Commit both the source and the generated JSON.

Debug a Block

1. Check the console for errors.
2. Compare expected vs actual DOM (custom decorators only add structure/classes).
3. Verify variant classes are applied (e.g. columns-band image-right/dark/teal).
4. For image issues, check the DM/Scene7 auto-block output in scripts.js.

Environments

Environment	URL Pattern	Purpose
Local	http://localhost:3000	Development
Feature Branch	https://{branch}-zurnelkay-eds-ue-oharris69.aem.page	PR testing
Preview	https://main-zurnelkay-eds-ue-oharris69.aem.page	Staging

Environment	URL Pattern	Purpose
Live	https://main-zurnelkay-eds-ue-oharris69.aem.live	Production

Git Workflow

Branch Naming

- Features: `feature/description`
- Fixes: `fix/description`
- Keep names short (branch names appear in preview URLs, which have length limits).

PR Requirements

1. Include the preview URL in the PR description.
2. Pass `npm run lint` (JS + CSS).
3. Test on the feature-branch preview, in both EN and FR where relevant.
4. No console errors.

Troubleshooting

Block not loading?

1. Confirm the block folder name matches the block class name in the DOM.
2. Verify the JS exports `default function decorate`.
3. Check the Network tab for 404s on the block JS/CSS.

Authoring model changes not showing in Universal Editor?

1. Confirm you edited `models/` and ran `npm run build:json`.
2. Confirm the regenerated `component-*.json` files were committed and published.

Images not rendering as responsive pictures?

1. Check whether the authored link is a recognized Dynamic Media / Scene7 URL (`isDMOpenAPIUrl`, `detectDynamicMediaUrl`).
2. Inspect the `<picture>` output from the DM auto-block in `scripts.js`.
3. Verify the Scene7 / DM viewer scripts in `head.html` are loading.

Styles not applying?

1. Check CSS specificity and that the token exists in `brand.css/styles.css`.
2. Verify the CSS file loads (Network tab).

Content not updating?

1. Clear browser cache; re-preview from AEM.
2. Wait for CDN cache (1–2 min).

Resources

Resource	URL
AEM Edge Delivery Services Documentation	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading ("Keeping it 100")	https://www.aem.live/developer/keeping-it-100
Universal Editor / crosswalk	https://www.aem.live/developer/component-model-definitions
Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices
Config Service (sites API)	https://www.aem.live/docs/admin.html#tag/siteConfig

Support Contacts

Config Service org admins (from the site config access block): - oharris69@gmail.com - oharris@adobe.com

[Add additional project-specific contacts as needed.]